

OOP in Java — Past Questions & Solutions

TU BCA 3rd Semester | Tribhuvan University

Years Covered: 2019 · 2020 · 2021 | Most Repeated Questions First

Subject	Object Oriented Programming in Java
Program	Bachelor in Computer Application (BCA)
Semester	Third Semester
University	Tribhuvan University (TU), Nepal
Structure	Group A: MCQ (10x1) Group B: 6x5=30 Group C: 2x10=20

■ **How to Use This Guide:** Section 1 lists the **MOST REPEATED** questions with full solutions — study these first as they have the highest chance of appearing in exams. Section 2 contains complete year-wise past papers (2019, 2020, 2021) with detailed answers.

■ TABLE OF CONTENTS

SECTION 1 MOST REPEATED QUESTIONS (Study These First!)

- 1.1 OOP Definition & Features [2019, 2020]
- 1.2 Abstract Class vs Interface [2019, 2020]
- 1.3 Exception Handling [2019, 2020, 2021]
- 1.4 Multiple Inheritance via Interface [2019, 2021]
- 1.5 Multithreading [2019, 2021]
- 1.6 Access Modifiers [2019, 2020]
- 1.7 File Handling [2019, 2021]
- 1.8 Method Overloading [2019, 2021]

SECTION 2 YEAR-WISE PAST QUESTIONS WITH SOLUTIONS

- 2.1 2019 Board Exam — Group B & C
- 2.2 2020 Board Exam — Group B & C
- 2.3 2021 Board Exam — Group B & C

SECTION 3 QUICK REVISION SUMMARY

■ SECTION 1: MOST REPEATED QUESTIONS — STUDY THESE FIRST!

The questions below appeared in **2 or more** TU board exams. Mastering these guarantees maximum marks in your exam.

Topic	Years Asked	Marks
OOP Definition & Features	2019, 2020	5
Abstract Class vs Interface	2019, 2020	5
Exception Handling	2019, 2020, 2021	5
Multiple Inheritance via Interface	2019, 2021	5
Multithreading (Runnable/Thread)	2019, 2021	5-10
Access Modifiers	2019, 2020	5
File Handling (Streams)	2019, 2021	5-10
Method Overloading + this keyword	2019, 2021	5

1.1 OOP Definition & Features [Asked: 2019, 2020]

Q: Define OOP. Explain the features of Object Oriented Programming Language. [5 Marks]

Answer:

OOP (Object-Oriented Programming) is a programming paradigm that organizes software design around **data (objects)** rather than functions and logic. Java is a fully object-oriented language.

Key Features of OOP:

1. Class & Object	Class is a blueprint/template. Object is a real-world instance created from a class. e.g., 'Car' is a class; 'myCar' is an object.
2. Encapsulation	Binding data and methods in a single unit (class) and hiding data from outside using access modifiers. e.g., private variables with public getters/setters.
3. Inheritance	A child class acquires properties and behaviors of a parent class using 'extends'. Promotes code reuse. e.g., Dog extends Animal.
4. Polymorphism	One interface, many forms. Achieved via method overloading (compile-time) and method overriding (runtime).
5. Abstraction	Hiding internal implementation and showing only essential features. Achieved via abstract classes and interfaces.
6. Message Passing	Objects communicate by calling each other's methods. This is how objects interact in a program.

1.2 Abstract Class vs Interface [Asked: 2019, 2020]

Q: Differentiate between abstract class and Interface with suitable example. [5 Marks]

Answer:

Feature	Abstract Class	Interface
Keyword	abstract	interface
Methods	Abstract + Concrete	Abstract only (before Java 8)
Variables	Any type	public static final only
Inheritance	extends (single)	implements (multiple allowed)
Constructor	Can have	Cannot have
Speed	Faster	Slightly slower

Example Program:

```
abstract class Vehicle {  
    abstract void speed();  
    void fuel() {  
    }  
}  
interface Electric {  
    void charge();  
}  
class Tesla extends Vehicle implements Electric {  
    public void speed() { System.out.println("Very fast!"); }  
    public void charge() { System.out.println("Charging battery..."); }  
    public static void main(String[] args) {  
        Tesla t = new Tesla();  
        t.speed();  
        t.fuel();  
        t.charge();  
    }  
}
```

■ **Output:**

```
Very fast!  
Uses fuel  
Charging battery...
```

1.3 Exception Handling [Asked: 2019, 2020, 2021 — MOST IMPORTANT!]

Q: Why is it important to handle exceptions? Write a program to illustrate exception handling. [5 Marks]

Answer:

An **exception** is an unexpected event that disrupts the normal flow of a program. Exception handling is important because:

- It prevents abnormal/abrupt termination of the program.
- It separates error-handling code from normal business logic.
- It provides meaningful error messages to users.
- It allows the program to recover and continue execution.

Keywords: try, catch, finally, throw, throws

```
class ExceptionExample {  
    public static void main(String[] args) {  
        // Example 1: ArithmeticException  
        try {  
            int a = 10, b = 0;  
            int result = a / b;  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Cannot divide by zero!");  
        } finally {  
            System.out.println("Finally block always runs.");  
        }  
        // Example 2: ArrayIndexOutOfBoundsException  
        try {  
            int[] arr = new int[3];  
            arr[10] = 5;  
        }  
    }  
}
```

```
0; // throws ArrayIndexOutOfBoundsException<br/> } catch (ArrayIndexOutOf
BoundsException e) {<br/> System.out.println("Error: Array index out of bounds!");<br/>
} }<br/> }<br/> }
```

■ Output:

```
Error: Cannot divide by zero!
Finally block always runs.
Error: Array index out of bounds!
```

Q: What are exception handling keywords? Write program for NullPointerException and NumberFormatException. [2021 — 5 Marks]

```
class ExceptionDemo {<br/> public static void main(String[] args) {<br/> // NullPoin
terException<br/> try {<br/> String s = null;<br/> System.out.pri
ntln(s.length()); // NullPointerException<br/> } catch (NullPointerException e) {<br/>
System.out.println("NullPointerException caught!");<br/> }<br/><br/>
// NumberFormatException<br/> try {<br/> int n = Integer.parseInt("abc"); /
/ NumberFormatException<br/> } catch (NumberFormatException e) {<br/> System
.out.println("NumberFormatException: Invalid number!");<br/> }<br/> }<br/> }
```

■ Output:

```
NullPointerException caught!
NumberFormatException: Invalid number!
```

1.4 Multiple Inheritance via Interface [Asked: 2019, 2021]

Q: How will you implement multiple inheritance using interface? Explain with program. [5 Marks]

Answer:

Java does **not** support multiple inheritance through classes (to avoid the diamond problem). However, it achieves multiple inheritance using **interfaces**. A class can implement multiple interfaces using the **implements** keyword.

```
interface Animal {<br/> void eat();<br/>}<br/>interface Bird {<br/> void fly();<br/>}<br/>
// Eagle implements BOTH interfaces = Multiple Inheritance<br/>class Eagle implements A
nimal, Bird {<br/> public void eat() { System.out.println("Eagle eats fish"); }<br/> pub
lic void fly() { System.out.println("Eagle flies high"); }<br/><br/> public static void mai
n(String[] args) {<br/> Eagle e = new Eagle();<br/> e.eat();<br/> e.fly()
;<br/> }<br/> }
```

■ Output:

```
Eagle eats fish
Eagle flies high
```

1.5 Multithreading [Asked: 2019, 2021]

Q: What is multithreading? Show how to create thread via Runnable interface and Thread class. [5 Marks]

Answer:

Multithreading is the process of executing two or more threads simultaneously. A **thread** is a lightweight sub-process, the smallest unit of processing. Java supports multithreading through the **Thread class** and **Runnable interface**.

```
// METHOD 1: Extending Thread class<br/>class MyThread1 extends Thread {<br/> public void r
un() {<br/> for (int i = 1; i <= 3; i++)<br/> System.out.println("Thread1: "
+ i);<br/> }<br/> }<br/><br/>// METHOD 2: Implementing Runnable interface<br/>class MyThrea
```

```

d2 implements Runnable {<br/>    public void run() {<br/>        for (int i = 1; i <= 3; i++)<br/>
br/>        System.out.println("Thread2: " + i);<br/>    }<br/>}<br/><br/>class ThreadDemo
{<br/>    public static void main(String[] args) {<br/>        MyThread1 t1 = new MyThread1()
;<br/>        t1.start();                // starts Thread1<br/><br/>        Thread t2
= new Thread(new MyThread2());<br/>        t2.start();                // starts Thre
ad2<br/>    }<br/>}}

```

■ Output:
Thread1: 1
Thread1: 2
Thread1: 3
Thread2: 1
Thread2: 2
Thread2: 3
(Order may vary)

Thread Lifecycle: New → Runnable → Running → Blocked/Waiting → Dead

1.6 Access Modifiers [Asked: 2019, 2020]

Q: Define access modifier. Explain access modifiers in Java with example. [5 Marks]

Answer:

Access modifiers control the **visibility/accessibility** of classes, methods, and variables.

Modifier	Same Class	Same Package	Subclass	Everywhere
private	YES	NO	NO	NO
default	YES	YES	NO	NO
protected	YES	YES	YES	NO
public	YES	YES	YES	YES

```

class Demo {<br/>    private    int x = 10;    // only inside class<br/>    int        y = 20;
    // package-level (default)<br/>    protected int z = 30;    // package + subclass<br/>    p
ublic    int w = 40;    // accessible everywhere<br/><br/>    public static void main(String[]
args) {<br/>        Demo d = new Demo();<br/>        System.out.println(d.x);    // 10 OK (same
class)<br/>        System.out.println(d.w);    // 40 OK (public)<br/>    }<br/>}

```

1.7 File Handling [Asked: 2019, 2021]

Q: Write a program to write and read from a file / copy content from one file to another. [5-10 Marks]

```

import java.io.*;<br/><br/>class FileDemo {<br/>    public static void main(String[] args) thr
ows IOException {<br/>        // WRITING to file<br/>        FileWriter fw = new FileWriter("s
tudent.txt");<br/>        fw.write("Roll: 1, Name: Ram, Address: Jhapa\n");<br/>        fw.wri
te("Roll: 2, Name: Sita, Address: Kathmandu\n");<br/>        fw.write("Roll: 3, Name: Hari, Ad
dress: Jhapa\n");<br/>        fw.close();<br/>        System.out.println("Data written to file
.");<br/><br/>        // READING from file<br/>        BufferedReader br = new BufferedReader(
new FileReader("student.txt"));<br/>        System.out.println("\nStudents from Jhapa:");<br/>
        String line;<br/>        while ((line = br.readLine()) != null) {<br/>            if (
line.contains("Jhapa"))<br/>                System.out.println(line);<br/>            }<br/>
        br.close();<br/>    }<br/>}

```

■ Output:

Data written to file.

Students from Jhapa:

Roll: 1, Name: Ram, Address: Jhapa

Roll: 3, Name: Hari, Address: Jhapa

1.8 Method Overloading + 'this' Keyword [Asked: 2019, 2021]

Q: What are uses of 'this' keyword? Write a program to implement method overloading. [5 Marks]

Uses of 'this' keyword:

- Refers to the current class instance variable (to avoid ambiguity).
- Calls the current class constructor using this().
- Passes current object as an argument to another method.

```
class Calculator {  
    int roll;  
    String name;  
  
    // 'this' used to differentiate instance var from parameter  
    Calculator(int roll, String name) {  
        this.roll = roll;  
        this.name = name;  
    }  
  
    // Method Overloading: same name, different parameters  
    int add(int a, int b) { return a + b; }  
    double add(double a, double b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator(1, "Ram");  
        System.out.println("Student: " + c.name);  
        System.out.println(c.add(3, 4)); // int version  
        System.out.println(c.add(2.5, 3.5)); // double version  
        System.out.println(c.add(1, 2, 3)); // three param version  
    }  
}
```

■ Output:

Student: Ram

7

6.0

6

■ SECTION 2: YEAR-WISE PAST QUESTIONS WITH SOLUTIONS

■ 2019 — TU BCA 3rd Semester Board Exam | OOP in Java

GROUP B — Attempt any SIX [6 x 5 = 30 Marks]

[2019 — Q2] Explain different types of control statements used in Java. [5 Marks]

Answer: Control statements control the flow of program execution.

Type	Statements	Use
Selection	if, if-else, switch	Decision making
Looping	for, while, do-while	Repetition
Jump	break, continue, return	Transfer control

```
// if-else<br>int n = 10;<br>if (n % 2 == 0) System.out.println("Even");<br>else<br>    System.out.println("Odd");<br><br>// for loop<br>for (int i = 1; i <= 5; i++)<br>    System.out.print(i + " ");    // 1 2 3 4 5<br><br>// switch<br>int day = 2;<br>switch(day) {<br>/>    case 1: System.out.println("Monday");    break;<br>    case 2: System.out.println("Tuesda<br>y");    break;<br>    default: System.out.println("Other");<br>}
```

[2019 — Q3] Define Abstract Class. Explain different types of Access Control. [5 Marks]

■ **Abstract Class and Access Modifiers** are covered in detail in Section 1.2 and Section 1.6 above. Please refer to those sections for complete answers.

[2019 — Q4] Define method Overriding. Write a program for multiple inheritance. [5 Marks]

Method Overriding: Redefining a parent class method in the child class with the **same name, return type, and parameters**. Used to achieve runtime polymorphism.

```
class Animal {<br>    void sound() { System.out.println("Animal makes sound"); }<br><br><br>}<br>class Dog extends Animal {<br>    // Overriding parent method<br>    @Override<br>    void sound() { System.out.println("Dog barks: Woof!"); }<br><br>    public static void main(String[] args) {<br>        Animal a = new Animal();<br>        a.sound();    // Animal<br>        makes sound<br><br>        Dog d = new Dog();<br>        d.sound();    // Dog barks:<br>        Woof! (overridden)<br><br>        Animal a2 = new Dog();    // Runtime Polymorphism<br>        a2.sound();    // Dog barks: Woof!<br>    }<br>}
```

■ **Output:**
Animal makes sound
Dog barks: Woof!
Dog barks: Woof!

■ **Multiple Inheritance via interface** is covered in Section 1.4.

[2019 — Q5] Why handle exceptions? Write a program illustrating exception handling. [5 Marks]

■ **Covered in detail in Section 1.3.**

[2019 — Q6] Define static keyword. Write four String methods with examples. [5 Marks]

static keyword: A static member belongs to the **class**, not to individual objects. It can be accessed without creating an object.

```
class Counter {  
    static int count = 0;    // shared among all objects  
    Counter()  
{ count++; }  
    public static void main(String[] args) {  
        new Counter();  
        new Counter();  
        System.out.println("Total objects: " + Counter.count);  
    }  
}
```

4 Important String Methods:

```
String s = "Hello World";  
System.out.println(s.length());    // 11  
System.out.println(s.toUpperCase());    // HELLO WORLD  
System.out.println(s.substring(6));    // World  
System.out.println(s.contains("Hello"));    // true  
System.out.println(s.replace("World", "Java"));    // Hello Java  
System.out.println(s.indexOf("World"));    // 6
```

GROUP C — Attempt any TWO [2 x 10 = 20 Marks]

[2019 — Q9a] Define Multithreading. Write a program for inter-thread communication. [10 Marks]

■ Basic multithreading with Runnable/Thread is covered in Section 1.5.

Inter-Thread Communication (Producer-Consumer):

```
class SharedResource {  
    int data;  
    boolean available = false;  
  
    synchronized void produce(int val) {  
        while (!available) {  
            try { wait(); }  
        } catch (InterruptedException e) {}  
        data = val;  
        System.out.println("Produced: " + data);  
        available = true;  
        notify();  
    }  
  
    synchronized void consume() {  
        while (!available) {  
            try { wait(); } catch (InterruptedException e) {}  
        }  
        System.out.println("Consumed: " + data);  
        available = false;  
        notify();  
    }  
}  
  
class Producer extends Thread {  
    SharedResource r;  
    Producer(SharedResource r) { this.r = r; }  
    public void run() { for(int i=1;i<=3;i++) r.produce(i); }  
}  
  
class Consumer extends Thread {  
    SharedResource r;  
    Consumer(SharedResource r) { this.r = r; }  
    public void run() { for(int i=1;i<=3;i++) r.consume(); }  
}  
  
class Main {  
    public static void main(String[] args) {  
        SharedResource r = new SharedResource();  
        new Producer(r).start();  
        new Consumer(r).start();  
    }  
}
```

■ Output:

```
Produced: 1  
Consumed: 1  
Produced: 2  
Consumed: 2  
Produced: 3  
Consumed: 3
```

[2019 — Q9b] Define Stream. Write a program to copy content from one file to another. [10 Marks]

Stream: A stream is a sequence of data. Java provides two types: **InputStream/OutputStream** (byte-based) and **Reader/Writer** (character-based).

```
import java.io.*;  
  
class FileCopy {  
    public static void main(String[] args) throws IOException {  
        // Writing to source file  
        FileWriter fw = new FileWriter("source.txt");  
        fw.write("This is TU BCA OOP Java Notes.");  
        fw.close();  
  
        // Copying: read from source, write to destination  
        FileInputStream in = new FileInputStream("source.txt");  
        FileOutputStream out = new FileOutputStream("dest.txt");  
        int ch;  
        while ((ch = in.read()) != -1) {  
            out.write(ch);  
            in.close();  
            out.close();  
        }  
        System.out.println("File copied successfully!");  
    }  
}
```

■ Output:

File copied successfully!

■ 2020 — TU BCA 3rd Semester Board Exam | OOP in Java

GROUP B — Attempt any SIX [6 x 5 = 30 Marks]

[2020 — Q2] Define OOP. Write the characteristics of OOP Language. [5 Marks]

■ Covered in detail in Section 1.1.

[2020 — Q3] Explain the Operators available in Java Programming. [5 Marks]

Answer:

Type	Operators	Example
Arithmetic	+, -, *, /, %	a + b
Relational	==, !=, >, <, >=, <=	a > b
Logical	&&, , !	a>0 && b>0
Bitwise	&, , ^, ~, <<, >>	a & b
Assignment	=, +=, -=, *=, /=	a += 5
Ternary	? :	a>b ? a : b
Unary	++, --	a++

[2020 — Q4] Define Loop. Write a Java program to print first N prime numbers. [5 Marks]

Loop: A control structure that repeats a block of code until a condition is met. Types: for, while, do-while.

```
class PrimeN {  
    public static void main(String[] args) {  
        int n = 10, count = 0, num = 2;  
        System.out.println("First " + n + " prime numbers:");  
        while (count < n) {  
            boolean isPrime = true;  
            for (int i = 2; i <= num / 2; i++) {  
                if (num % i == 0) {  
                    isPrime = false; break;  
                }  
            }  
            if (isPrime) {  
                System.out.print(num + " ");  
                count++;  
            }  
            num++;  
        }  
    }  
}
```

■ Output:

```
First 10 prime numbers:  
2 3 5 7 11 13 17 19 23 29
```

[2020 — Q5] Differentiate between abstract class and Interface. [5 Marks]

■ Covered in detail in Section 1.2.

[2020 — Q6] Define access modifier. Explain access modifiers in Java. [5 Marks]

■ Covered in detail in Section 1.6.

[2020 — Q7] Define exception. Explain exception handling mechanism in Java. [5 Marks]

■ Covered in detail in Section 1.3.

[2020 — Q8] Write short notes on: (a) final keyword (b) Collection class (c) JDBC [5 Marks]

(a) final keyword:

- final variable: value cannot be changed (constant).
- final method: cannot be overridden by subclass.
- final class: cannot be inherited/extended.

(b) Collection class: Java Collections Framework provides ready-made data structures. Key interfaces: List (ArrayList, LinkedList), Set (HashSet, TreeSet), Map (HashMap, TreeMap).

(c) JDBC (Java Database Connectivity): An API that allows Java programs to connect and interact with relational databases. Steps: Load Driver → Connect → Create Statement → Execute Query → Close.

GROUP C — Attempt any TWO [2 x 10 = 20 Marks]

[2020 — Q9a] Write a program to create and use Java package. [5 Marks]

```
// File: mypackage/Greet.java<br/>package mypackage;<br/>public class Greet {<br/>    public void hello(String name) {<br/>        System.out.println("Hello, " + name + "! Welcome to TU BC A.");<br/>    }<br/>}<br/>// File: Main.java<br/>import mypackage.Greet;<br/>class Main {<br/>    public static void main(String[] args) {<br/>        Greet g = new Greet();<br/>        g.hello("Ram");<br/>    }<br/>}
```

■ **Output:**

Hello, Ram! Welcome to TU BCA.

[2020 — Q10a] Write a program to sort names of five cities in ascending order. [5 Marks]

```
import java.util.Arrays;<br/>class SortCities {<br/>    public static void main(String[] args) {<br/>        String[] cities = {"Pokhara", "Kathmandu", "Biratnagar", "Lalitpur", "Dharan"};<br/>        Arrays.sort(cities);<br/>        System.out.println("Cities in Ascending Order:");<br/>        for (String city : cities)<br/>            System.out.println(city);<br/>    }<br/>}
```

■ **Output:**

Cities in Ascending Order:

Biratnagar
Dharan
Kathmandu
Lalitpur
Pokhara

[2020 — Q10b] Define Polymorphism. How do we achieve polymorphism in Java? [5 Marks]

Polymorphism means 'many forms'. It allows one interface to be used for many data types.

Type	Compile-time Polymorphism	Runtime Polymorphism
Also called	Static/Early binding	Dynamic/Late binding
Achieved via	Method Overloading	Method Overriding
Resolved at	Compile time	Runtime

■ 2021 — TU BCA 3rd Semester Board Exam | OOP in Java

GROUP B — Attempt any SIX [6 x 5 = 30 Marks]

[2021 — Q2] What is jagged array? Write a program to initialize and display jagged array with row sums. [5 Marks]

Jagged Array: A multidimensional array where each row can have a **different number of columns**. Also called a **ragged array**.

```
class JaggedArray {  
    public static void main(String[] args) {  
        int[][] jag =  
            new int[3][]; // 3 rows, columns vary  
        jag[0] = new int[]{1, 2, 3}; // row 0: 3 columns  
        jag[1] = new int[]{4, 5}; // row 1: 2 columns  
        jag[2] = new int[]{6, 7, 8, 9}; // row 2: 4 columns  
        for (int i = 0; i < jag.length; i++) {  
            int sum = 0;  
            System.out.print("Row " + i + ": ")  
            for (int j = 0; j < jag[i].length; j++) {  
                System.out.print(jag[i][j] + " ");  
                sum += jag[i][j];  
            }  
            System.out.println(" => Sum = " + sum);  
        }  
    }  
}
```

■ Output:

```
Row 0: 1 2 3 => Sum = 6  
Row 1: 4 5 => Sum = 9  
Row 2: 6 7 8 9 => Sum = 30
```

[2021 — Q3] What are uses of this keyword? Write a program to implement method overloading. [5 Marks]

■ Covered in detail in Section 1.8.

[2021 — Q4] Why is inheritance needed in Java? Implement multiple inheritance using interface. [5 Marks]

Why inheritance is needed:

- Promotes code reuse — write once, use many times.
- Reduces code redundancy and maintenance effort.
- Establishes IS-A relationship between classes.
- Supports runtime polymorphism (method overriding).

■ Multiple inheritance program is in Section 1.4.

[2021 — Q5] What is multithreading? Create thread via Runnable interface and Thread class. [5 Marks]

■ Covered in detail in Section 1.5.

[2021 — Q6] Exception handling keywords? Program for NullPointerException and NumberFormatException. [5 Marks]

■ Covered in detail in Section 1.3 (second part).

[2021 — Q7] What is serialization? Write program to write student info to file and display students from Jhapa. [5 Marks]

Serialization: The process of converting an object's state into a byte stream so it can be saved to a file or sent over a network. The class must implement **Serializable** interface.

```
import java.io.*;<br><br>class FileStudents {<br>    public static void main(String[] args)
    throws Exception {<br>        // Write student data<br>        PrintWriter pw = new PrintWr
iter(new FileWriter("students.txt"));<br>        pw.println("1,Ram Prasad,Jhapa,TU College");
<br>        pw.println("2,Sita Sharma,Kathmandu,PU College");<br>        pw.println("3,Hari
Bahadur,Jhapa,TU College");<br>        pw.close();<br><br>        // Read and filter by Jha
pa<br>        BufferedReader br = new BufferedReader(new FileReader("students.txt"));<br>
        System.out.println("Students from Jhapa:");<br>        String line;<br>        while ((
line = br.readLine()) != null) {<br>            String[] data = line.split(",");<br>
            if (data[2].equals("Jhapa")) {<br>                System.out.println("Roll: "+data[0]+",
Name: "+data[1]+", College: "+data[3]);<br>            }<br>        }<br>        br.close()
;<br>    }<br>}
```

■ Output:

Students from Jhapa:

Roll: 1, Name: Ram Prasad, College: TU College

Roll: 3, Name: Hari Bahadur, College: TU College

[2021 — Q8] What is Wrapper class? Explain iterator and comparator in collection objects. [5 Marks]

Wrapper Class: Converts primitive data types into objects. e.g., int → Integer, char → Character, double → Double. Used with Collections (which can only store objects).

```
import java.util.*;<br><br>class WrapperDemo {<br>    public static void main(String[] args)
    {<br>        // Wrapper class<br>        int x = 10;<br>        Integer obj = Integer.val
ueOf(x);        // boxing<br>        int back = obj.intValue();        // unboxing
<br>        System.out.println("Wrapper: " + obj);<br><br>        // Iterator<br>        A
rrayList list = new ArrayList();<br>        list.add("Banana"); list.add("Apple"); list.add("
Mango");<br>        Iterator it = list.iterator();<br>        System.out.print("Iterator: ")
;<br>        while (it.hasNext())<br>            System.out.print(it.next() + " ");<br><br>
>        // Comparator: sort by string length<br>        list.sort(Comparator.comparingInt(o
-> o.toString().length()));<br>        System.out.println("\nSorted by length: " + list);<br>
>    }<br>}
```

■ Output:

Wrapper: 10

Iterator: Banana Apple Mango

Sorted by length: [Apple, Mango, Banana]

GROUP C — Attempt any TWO [2 x 10 = 20 Marks]

[2021 — Q9] Class vs Interface. Create a Time class with add/subtract/display methods. [10 Marks]

```
class Time {<br>    int hours, minutes, seconds;<br><br>    Time(int h, int m, int s) {<br>
>        hours = h; minutes = m; seconds = s;<br>    }<br><br>    void display() {<br>
        System.out.printf("Time: %02d:%02d:%02d\n", hours, minutes, seconds);<br>    }<br><br>
    Time add(Time t) {<br>        int totalSec = (this.hours*3600 + this.minutes*60 + this.sec
onds)<br>        + (t.hours*3600 + t.minutes*60 + t.seconds);<br>        return
        new Time(totalSec/3600, (totalSec%3600)/60, totalSec%60);<br>    }<br><br>
    Time subtrac
t(Time t) {<br>        int totalSec = (this.hours*3600 + this.minutes*60 + this.seconds)<br>
        - (t.hours*3600 + t.minutes*60 + t.seconds);<br>        if (totalSec < 0
) totalSec = 0;<br>        return new Time(totalSec/3600, (totalSec%3600)/60, totalSec%60);<b
r>    }<br><br>
    public static void main(String[] args) {<br>        Time t1 = new Time(
2, 30, 45);<br>        Time t2 = new Time(1, 20, 30);<br>        t1.display();<br>        t
2.display();<br>        Time added = t1.add(t2);<br>        added.display();        //
        result of addition<br>        Time subbed = t1.subtract(t2);<br>        subbed.display();
        // result of subtraction<br>    }<br>}
```

■ Output:

Time: 02:30:45

Time: 01:20:30

Time: 03:51:15

Time: 01:10:15

■ SECTION 3: QUICK REVISION & EXAM TIPS

Marks Distribution by Topic

Topic	Expected Marks	Priority
OOP Concepts & Features	5-10	HIGH
Constructors & Methods	5-10	HIGH
Inheritance & Polymorphism	10-12	VERY HIGH
Interfaces & Abstract Class	8-10	VERY HIGH
Exception Handling	8-10	VERY HIGH
Multithreading	5-10	HIGH
File Handling	5-10	HIGH
Collections & Wrapper	5	MEDIUM
Packages	5	MEDIUM
AWT/Swing/JDBC	10	MEDIUM

Exam Strategy & Tips

Group A (MCQ)	10 questions x 1 mark = 10. Attempt ALL. No negative marking.
Group B (5 Marks)	Attempt any 6 out of 8. Write: Definition (1) + Program/Explanation (4). Always include output.
Group C (10 Marks)	Attempt any 2 out of 3. Write: Theory (3-4 marks) + Full working program with output (6-7 marks).
Programs	Always write OUTPUT after every program. TU gives separate marks for correct output.
Definitions	Learn exact definitions — short 2-mark sub-questions are easy marks.
Repeated Topics	OOP features, Exception Handling, Inheritance, Multithreading repeat EVERY year. Prepare these thoroughly.
Time Management	Spend 15 min on MCQ, 45 min on Group B, 60 min on Group C.

■ This PDF covers TU BCA 3rd Semester OOP in Java past questions from 2019, 2020, and 2021. Section 1 contains the most repeated questions — read those first! For 2022, 2023, and 2024 questions, refer to official TU resources at bcapoint.com or bcanepaltu.com. Best of luck with your exams! ■